

User Manual for the VSIIE solvers

September 17, 2026

Abstract

This document provides a user manual for a software framework that implements different Variable Stepsize Implicit-Implicit-Explicit (VSIIE) solvers. The Implicit-Implicit-Explicit (IIE) solvers [3] are a family of high-order time integration methods designed to efficiently and accurately solve stiff systems of ordinary differential equations (ODEs) that often arise from the spatial discretization of relevant models based on partial differential equations (PDEs) such as diffusion-reaction-advection (DRA) models, which can be divided into three separate terms. Here, the IIE solvers are extended to allow for variable stepsize, resulting in the VSIIE solvers. In addition, an implementation of Variable Stepsize Semi-implicit Backward Differentiation Formula (VSSBDF) solvers [1] is also included for the purposes of experimental comparison.

1 User manual

The user manual describes how to use the software framework and several `Python` scripts developed to perform automatically experiments to compare the performance of the fixed stepsize IIE methods [3], the variable stepsize VSSBDF methods [1], and the variable stepsize VSIIE methods [2].

1.1 Building and Calling the solver

The executable of the solver can be generated by typing `make` in the main folder (usually named `VSIIE`) where the source code is located. This will generate the executable file called `IVP_Solver` in the same folder.

The solver can be called from the command line in a terminal window. The call follows the following format:

IVP_Solver *Solver problem_id. order Neqn tf stepsize tol num_reps alpha eta_min eta_max*

where:

- *Solver*: It denotes the type of solver to be used. The provided values can be:
 - `IIE`: A constant stepsize IIE method is used.
 - `VSIIE`: A Variable Stepsize IIE (VSIIE) method is used.
 - `VSSBDF`: A Variable Stepsize Semiimplicit BDF (VSSBDF) method is used.
- *problem_id.*: It denotes the mathematical model to be solved. The provided values can be:

- 0: 1D Advection-Diffusion.
 - 1: Stiff Brusselator.
 - 2: FKPP Combustion.
 - 3: Ignition Combustion.
 - 4: Fisher Combustion.
 - 5: High Irradiance RESponse (HIRES) problem [4].
- *order*: It denotes the order of the VSIIE/IIE method to be used (1, 2, 3 or 4).
 - *Neqn*: It denotes the number of ODEs in the system to be solved (It is important not to confuse this with the number of grid points.).
 - *tf*: It denotes the final simulation time.
 - *stepsize*: It denotes the constant stepsize if const_var=const (IIE method) and the initial stepsize if const_var=var (VSIIE method)
 - *tol*: Error tolerance which it is only used for VSIIE (const_var=var)
 - *num_reps*: Number of repetitions of the experiment (the experiment is repeated several times in order to obtain a report the minimum runtime of all repetitions).
 - *alpha eta_min eta_max* (Optional): Parameters α , $\eta_{\min}$ and $\eta_{\max}$ of the adaptive time-stepping strategy (by default: $\alpha = 0.8$, $\eta_{\min} = 0.5$ and $\eta_{\max} = 4$).

Therefore, each call to the solver is applied to solve a particular IVP model instance using a particular method (VSIIE, VSSBDF or IIE) of a given order.

When the solver is called, generates output by the screen and in output files. The first time the solver is called for a particular problem and final time, it creates a folder called `data` in the same folder where the executable is located and generates a text file where the reference solution for this problem instance is saved. This reference solution is computed by using the VSSBDF method of order 4 with a very small tolerance of 10^{-6} . Thus, the reference solution data can be reused in subsequent calls of the solver for the same problem instance, avoiding recomputation.

After running the solver, it generates a line with data in a text output file with `.txt` extension, whose name depends on the particular IVP model, method and final time. This output file is located in the same folder of the executable. The output file contains the following data in each line (separated by several spaces):

- **For VSIIE and VSSBDF**: [tolerance] [error] [Minimum runtime] [Number of time steps] [Number of rejected time steps] [Number of accepted time steps] [Average number of Newton iterations per implicit solve]
- **For IIE**: [Constant stepsize] [Error] [Minimum runtime] [Number of time steps] [Number of rejected time steps=0] [Number of accepted time steps] [Average number of Newton iterations per implicit solve]

1.2 The scripts

There is a `Python` script provided to run automatically the experiments corresponding to each particular mathematical IVP model and generate the corresponding plots (work-precision and time-steps). Each script runs the experiments for the IIE, VSSBDF and VSIIE methods using different parameters. Each `Python` script must be called in the particular folder where the script is located (although the user can call it from other locations if he creates a new folder, copies the script to the new folder and specifies what is the name of the new folder in the script). The location and correspondin model of the particular scripts are given in Table 1.

Table 1: Data about the particular python scripts

Script Name	Folder	Model
<code>adr.py</code>	ADR	1D Advection-Diffusion
<code>sbr.py</code>	SBR	Stiff Brusselator
<code>fkpp.py</code>	FKPP	FKPP Combustion
<code>ignition.py</code>	IGNIT	Ignition Combustion
<code>fisher.py</code>	FISHER	Fisher Combustion
<code>hires.py</code>	HIRES	High Irradiance RESponse (HIRES) problem

For each `Python` script, the user must define the following variables which define the experiments to be run:

- **Problem:** It denotes the problem id (0 to 4) corresponding to the mathematical IVP model.
- **IVP_name:** It denotes the name of the mathematical IVP model. The provided values can be:
 - "1D_Adv-Diff": 1D Advection-Diffusion.
 - "stiff_Brusselator": Stiff Brusselator.
 - "FKPP": FKPP Combustion.
 - "Ignition": Ignition Combustion.
 - "Fisher": Fisher Combustion.
 - HIRES: High Irradiance RESponse (HIRES) problem.
- **Neqn:** It denotes the number of ODEs in the system to be solved (It must be the the number of equations in the chosen IVP model, not the number of grid points).
- **Tf:** It denotes the final simulation time.
- **FOLDER:** It denotes the folder name where the experiments will be run (It must be the folder where the chosen `Python` script is located). For instance, "ADR" for the 1D Advection-Diffusion model.
- **stepsize_array:** It denotes the 2D array of constant stepsizes to be used for the IIE methods of different orders. It is also used to define the initial stepsizes for the VSIIE methods of different orders. For instance, `stepsize_array[0]` contains the array of constant stepsizes for the fisrst order IIE method and the initial stepsizes for the first order VSIIE method.

- **VSIIE_tol_array**: It denotes the array of error tolerances to be used for the VSIIE methods of different orders. For ADR, the definition of this array can be as follows:

```
stepsize_array = [
    [7.5e-3, 1e-3, 5e-4, 1.0e-4, 5e-5, 1e-5, 5e-6, 1e-6, 5e-7, 1e-7], #order1
    [5e-2, 2.5e-2, 1e-2, 7.5e-3, 5e-3, 1e-3, 5e-4, 1e-4, 5e-5, 1e-5, 5e-6, 1e-6], #order2
    [1e-1, 2.5e-2, 1e-2, 7.5e-3, 5e-3, 1e-3, 5e-4, 1e-4], #order3
    [1e-1, 7.5e-2, 5e-2, 2.5e-2, 1e-2, 7.5e-3, 5e-3, 1e-3, 5.0e-4]] #order4
```

- **VSSBDF_tol_array**: It denotes the array of error tolerances to be used for the VSSBDF methods of different orders. It is defined in the same way as VSIIE_tol_array.
- **alpha_array**: It denotes the array of values which indicates the α parameter values (which affects to the time-stepping algorithm) to be used for each particular experiment of the VSIIE methods of different orders. For ADR, the definition of this array can be as follows:

```
alpha_array = [
    [0.8, 0.8, 0.8, 0.8, 0.8, 0.8, 0.8, 0.8, 0.8, 0.8], #order1
    [0.4, 0.4, 0.4, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6, 0.6], #order2
    [0.3, 0.3, 0.4, 0.4, 0.4, 0.4, 0.4, 0.4], #order3
    [0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3, 0.3]] #order4
```

The arrays `stepsize_array`, `VSIIE_tol_array` and `alpha_array` must have the same number of elements for each order.

1.3 Calling the Python scripts

Each Python script can be called from the command line in a terminal window (being on the folder where the script is located). The call has the following format:

```
script_name.py run_type min_order max_order IIE_min_order IIE_max_order N_Reps
```

where:

- **run_type**: It denotes if the experiments are to be run or not. The provided values can be:
 - 0: Do not run the experiments but only plot them from the previously obtained data files.
 - 1: Run the experiments, save the results to data files and then plot them.
- **min_order max_order**: Minimum and maximum order of the corresponding VSIIE/VSSBDF family of methods to be used in the experiments.
- **IIE_min_order IIE_max_order**: Minimum and maximum order of the IIE methods to be used in the experiments.
- **N_Reps**: Number of repetitions of each experiment to obtain the minimum runtime.

If the user wish to run the experiments for all the orders of all VSIIE, VSSBDF and IIE methods with 3 repetitions, he can call the script as follows:

script_name.py 1 1 4 1 4 3.

On the other hand, if the user wish to run the experiments for a particular range of orders of VSIE and/or IE methods, he can specify the corresponding minimum and maximum orders in the call to the script. So, for example, the following call

script_name.py 1 2 3 1 2 4

runs the experiments for orders 2 and 3 of the VSIE/VSSBDF methods and orders 1 and 2 of the IE methods using 4 repetitions.

One can also avoid easily to do experiments with a particular method (IE or VSIE). So for example, the the call:

script_name.py 1 2 2 0 0 3

only runs the experiments for order 2 of the VSIE methods (there is no experiment for IE methods). In this way, the scripts allows the user to choose what orders to be used in the experiments and thus avoid to recompute data which have been already obtained previously.

References

- [1] Raed Ali Mara'Beh, J.M. Mantas, P. González, and Raymond J. Spiteri. Performance comparison of variable-stepsize IMEX SBDF methods on Advection-Diffusion-Reaction models. *Computers & Mathematics with Applications*, 190:41–56, 2025.
- [2] Raed Ali Mara'Beh, J.M. Mantas, and Raymond J. Spiteri. Variable-stepsize 3-additive linear multistep methods on Diffusion-Reaction-Advection models. *Submitted to IEEE Access*, 2026.
- [3] Raed Ali Mara'Beh, Raymond J. Spiteri, P. González, and José M. Mantas. 3-additive linear multi-step methods for Diffusion-Reaction-Advection models. *Applied Numerical Mathematics*, 183:15–38, 2023.
- [4] E. Schäfer. A new approach to explain the "high irradiance responses of photomorphogenesis on the basis of phytochrome. *Journal of Mathematical Biology*, 2:41–56, 1975.